



**Universidad
Gerardo Barrios**

Carrera:

Ingeniería en Sistema y Redes Informáticas.

Asignatura:

Programación Computacional 4

Tema Para Presentar:

Laboratorio 2 Tercera Parte Del Computo 3

Docente:

Ing. William Alexis Montes Girón

Presentado por:

Rene Oswaldo Orellana de la O SMSS018422

Allisson Lourdes Guevara Palma SMIS038421

Introducción:

Corrección del Sistema de Enrutamiento (Fix de Errores 404)

El servidor arrojaba errores 404 Not Found debido a dos inconsistencias críticas en la arquitectura de Laravel y el manejo de rutas en el Frontend:

Configuración del Servidor (routes/web.php): El archivo de rutas original solo tenía registradas las URLs para /, /login y /mapas. Se añadieron formalmente las rutas faltantes para mapear correctamente las vistas de `historial.blade.php` y `avisodeprivacidad.blade.php`.

Absolutización de Enlaces en JavaScript (Navbar.js y Footer.js): Los componentes inyectados dinámicamente usaban rutas relativas (ej. `href="login"`), lo que provocaba que al navegar desde una subpágina el navegador concatenara erróneamente las URLs (ej. `/login/avisodeprivacidad`). Se corrigieron todos los enlaces anteponiendo una diagonal (/) para convertirlos en rutas absolutas (ej. `href="/login"`), permitiendo una navegación fluida desde cualquier sección.

Limpieza de Caché: Se ejecutó el comando de Artisan para limpiar el historial de rutas optimizadas en el framework, forzando a Laravel a reconocer la nueva distribución de inmediato.

Rediseño de Componentes Visuales y Reemplazo de Assets

Se eliminó la dependencia de gráficos vectoriales complejos en código y se estandarizaron los contenedores para dar una identidad visual uniforme:

Migración de SVG a PNG: Se removieron por completo los bloques de código puro que generaban la ilustración geométrica del círculo con montañas rosadas. En su lugar, se implementó la etiqueta nativa apuntando al nuevo asset gráfico estático `montaña1.png` guardado en la carpeta `public/assets/`.

Aplicación de este cambio: La sustitución e integración de la nueva imagen se realizó de forma idéntica tanto en la interfaz de `login.blade.php` como en la tarjeta interna de `avisodeprivacidad.blade.php`, logrando consistencia en todo el módulo de usuario.

Optimización de Estilos con Tailwind CSS

Se depuró el marcado HTML para cumplir con las directrices de optimización del compilador de Tailwind en VS Code y mejorar la experiencia de usuario:

Resolución de Advertencias del Editor (Clean Code): Se reemplazaron las clases de tamaño arbitrario manuales como `max-w-[260px]`, las cuales generaban alertas amarillas en el entorno de desarrollo, por las clases nativas estándar de la escala oficial: `max-w-64` y `max-h-64` (`256\text{ px}`).

Efectos Interactivos (Microinteracciones): Se añadieron las clases `transition-transform duration-500 hover: scale-105` a la imagen de la montaña, permitiendo un sutil escalado animado del 5% cuando el usuario pasa el cursor sobre ella.

Ajuste de Sombras y Centrado: Se incorporó la clase `drop-shadow-2xl` para proyectar un relieve suave sobre los fondos translúcidos con `glassmorphism` de las tarjetas y se forzó el centrado absoluto mediante contenedores en modo `flex items-center justify-center`

Introducción:.....	1
Descripción de los módulos implementados en este tercer avance y los que aún faltan por desarrollar.	4
Descripción de las diferentes pruebas que se implementarán para verificar el funcionamiento y estado del sistema.	5
Evidencias de trabajo grupal	7

Descripción de los módulos implementados en este tercer avance y los que aún faltan por desarrollar.

La aplicación web **SkyCast** es un sistema de monitoreo climático desarrollado con **Laravel en el backend** y **JavaScript asíncrono en el frontend**, organizado en varios módulos principales:

1. **Autenticación y seguridad**
Gestiona el acceso de usuarios con inicio de sesión y registro.
2. **Consulta climatológica (núcleo)**
Es el módulo principal que obtiene datos del clima en tiempo real (usando OpenWeather), permite buscar ciudades y muestra información como temperatura, humedad y viento.
3. **Pronóstico extendido**
Ofrece predicciones del clima a 5 días con visualización de cambios de temperatura.
4. **Historial de búsquedas**
Guarda las consultas realizadas por el usuario, permite acceder rápidamente a ellas y eliminarlas.
5. **Interfaz geográfica (mapas)**
Muestra mapas interactivos con capas climáticas como lluvia, viento y nubes.
6. **Soporte y legales**
Incluye contacto para ayuda técnica y políticas de privacidad sobre el manejo de datos.
7. **Componentes globales**
Elementos reutilizables como navegación, pie de página y estilos, que mantienen coherencia en toda la app

Descripción de las diferentes pruebas que se implementarán para verificar el funcionamiento y estado del sistema.

1. Pruebas Unitarias (Unit Testing)

Se enfocan en verificar que las funciones más pequeñas e individuales del código hagan exactamente lo que se espera de ellas, de forma aislada.

Prueba de formateo de datos climáticos: Verificar que las funciones en `WeatherServices.js` procesen correctamente la respuesta cruda de OpenWeather (por ejemplo, convertir grados Kelvin a Celsius o metros por segundo a km/h).

Prueba de validación de entradas del Login: Validar que los scripts del formulario de inicio de sesión rechacen estructuras de correo inválidas o campos vacíos antes de enviar la petición.

Herramientas sugeridas: PHPUnit (integrado en Laravel) para componentes lógicos del backend y Jest o Vitest para las funciones de JavaScript en `public/js/`.

2. Pruebas de Integración (Integration Testing)

Estas pruebas verifican que los diferentes módulos o componentes del sistema interactúen entre sí correctamente y sin romper la aplicación.

Integración Frontend-API (Consumo de OpenWeather): Validar que cuando `main.js` invoca a `WeatherServices.js`, los datos recibidos actualicen correctamente los elementos del DOM en `index.blade.php` en lugar de quedarse congelados en "Cargando...".

Integración de Vistas y Enrutamiento: Comprobar que el archivo `routes/web.php` despache la vista HTML correcta (login, mapas, historial, avisodeprivacidad) cuando el usuario hace clic en los enlaces absolutos del componente dinámico `Navbar.js`.

3. Pruebas de Interfaz de Usuario y Navegación (UI / Funcionales)

Verifican el comportamiento del sistema desde la perspectiva del usuario final, asegurando que la experiencia visual y la interactividad sean óptimas.

Prueba de Enlaces y Navegación: Recorrer sistemáticamente la barra de navegación (`Navbar.js`) y el pie de página (`Footer.js`) desde cada una de las subpáginas para garantizar que no se vuelvan a generar rutas relativas rotas que rompan en errores 404.

Prueba de Animaciones e Imágenes: Confirmar que la imagen `montaña1.png` cargue correctamente en los paneles izquierdos de Login y Privacidad, y que responda de manera fluida al efecto visual de escalado (`hover:scale-105`).

Prueba de Responsividad (Mobile First): Verificar que el menú hamburguesa desplegable en Navbar.js funcione al hacer clic en pantallas táctiles y dispositivos móviles, y que las tarjetas se adapten usando Tailwind CSS sin desbordarse.

4. Pruebas de Sistema y Entorno (Environment Testing)

Dedicadas a validar el comportamiento del software cuando se expone a redes externas, túneles de desarrollo o servidores de producción.

Prueba de Geolocalización en entornos remotos: Verificar la API de geolocalización del navegador tanto en entorno local (127.0.0.1) como mediante entornos compartidos por Dev Tunnels (.devtunnels.ms), asegurando que los navegadores soliciten correctamente los permisos de ubicación al usuario.

Prueba de Asincronismo y Concurrencia en Túneles: Comprobar el comportamiento del flujo de datos cuando el puerto 8000 de VS Code está configurado en visibilidad Pública, garantizando que el túnel intermedio de Microsoft no bloquee las peticiones asíncronas de fondo (XHR/Fetch).

5. Pruebas de Caja Negra (Escenarios de Usuario)

Simulaciones de comportamiento real para comprobar la resistencia del sistema ante entradas inesperadas o fallos de red.

Escenario de Error de Red (Modo Offline): Desconectar el internet del servidor/cliente y verificar que la aplicación muestre un mensaje de alerta amigable (ej. "No se pudo conectar con el servicio meteorológico") en lugar de dejar la pantalla colgada indefinidamente.

Escenario de Búsqueda Inexistente: Introducir caracteres aleatorios o nombres de ciudades que no existen en la barra de búsqueda y validar que el sistema devuelva un estado controlado (ej. "Ciudad no encontrada") protegiendo la estabilidad del frontend.

Evidencias de trabajo grupal



